Chapter 1. Introduction

Most free software projects fail.

We tend not to hear very much about the failures. Only successful projects attract attention, and there are so many free software projects in total[1] that even though only a small percentage succeed, the result is still a lot of visible projects. We also don't hear about the failures because failure is not an event. There is no single moment when a project ceases to be viable; people just sort of drift away and stop working on it. There may be a moment when a final change is made to the project, but those who made it usually didn't know at the time that it was the last one. There is not even a clear definition of when a project is expired. Is it when it hasn't been actively worked on for six months? When its user base stops growing, without having exceeded the developer base? What if the developers of one project abandon it because they realized they were duplicating the work of another—and what if they join that other project, then expand it to include much of their earlier effort? Did the first project end, or just change homes?

Because of such complexities, it's impossible to put a precise number on the failure rate. But anecdotal evidence from over a decade in open source, some casting around on SourceForge.net, and a little Googling all point to the same conclusion: the rate is extremely high, probably on the order of 90–95%. The number climbs higher if you include surviving but dysfunctional projects: those which *are* producing running code, but which are not pleasant places to be, or are not making progress as quickly or as dependably as they could.

This book is about avoiding failure. It examines not only how to do things right, but how to do them wrong, so you can recognize and correct problems early. My hope is that after reading it, you will have a repertory of techniques not just for avoiding common pitfalls of open source development, but also for dealing with the growth and maintenance of a successful project. Success is not a zero-sum game, and this book is not about winning or getting ahead of the competition. Indeed, an important part of running an open source project is working smoothly with other, related projects. In the long run, every successful project contributes to the well-being of the overall, worldwide body of free software.

It would be tempting to say that free software projects fail for the same sorts of reasons proprietary software projects do. Certainly, free software has no monopoly on unrealistic requirements, vague specifications, poor resource management, insufficient design phases, or any of the other hobgoblins already well known to the software industry. There is a huge body of writing on these topics, and I will try not to duplicate it in this book. Instead, I will attempt to describe the problems peculiar to free software. When a free software project runs aground, it is often because the developers (or the managers) did not appreciate the unique problems of open source software development, even though they might have been quite prepared for the better-known difficulties of closed-source development.

One of the most common mistakes is unrealistic expectations about the benefits of open source itself. An open license does not guarantee that hordes of active developers will suddenly volunteer their time to your project, nor does open-sourcing a troubled project automatically cure its ills. In fact, quite the opposite: opening up a project can add whole new sets of complexities, and cost *more* in the short term than simply keeping it in-house. Opening up means arranging the code to be comprehensible to complete strangers, setting up a development web site and email lists, and often writing documentation for the first time. All this is a lot of work. And of course, if any interested developers *do* show up, there is the added burden of answering their questions for a while before seeing any benefit from their presence. As developer Jamie Zawinski said about the troubled early days of the Mozilla project:

> *Open source does work, but it is most definitely not a panacea. If there's a cautionary*

[1] SourceForge.net, one popular hosting site, had 79,225 projects registered as of mid-April 2004. This is nowhere near the total number of free software projects on the Internet, of course; it's just the number that chose to use SourceForge.

> *tale here, it is that you can't take a dying project, sprinkle it with the magic pixie dust of "open source," and have everything magically work out. Software is hard. The issues aren't that simple.*
>
> (from **http://www.jwz.org/gruntle/nomo.html**)

A related mistake is that of skimping on presentation and packaging, figuring that these can always be done later, when the project is well under way. Presentation and packaging comprise a wide range of tasks, all revolving around the theme of reducing the barrier to entry. Making the project inviting to the uninitiated means writing user and developer documentation, setting up a project web site that's informative to newcomers, automating as much of the software's compilation and installation as possible, etc. Many programmers unfortunately treat this work as being of secondary importance to the code itself. There are a couple of reasons for this. First, it can feel like busywork, because its benefits are most visible to those least familiar with the project, and vice versa. After all, the people who develop the code don't really need the packaging. They already know how to install, administer, and use the software, because they wrote it. Second, the skills required to do presentation and packaging well are often completely different from those required to write code. People tend to focus on what they're good at, even if it might serve the project better to spend a little time on something that suits them less. Chapter 2, *Getting Started* discusses presentation and packaging in detail, and explains why it's crucial that they be a priority from the very start of the project.

Next comes the fallacy that little or no project management is required in open source, or conversely, that the same management practices used for in-house development will work equally well on an open source project. Management in an open source project isn't always very visible, but in the successful projects, it's usually happening behind the scenes in some form or another. A small thought experiment suffices to show why. An open source project consists of a random collection of programmers—already a notoriously independent-minded category—who have most likely never met each other, and who may each have different personal goals in working on the project. The thought experiment is simply to imagine what would happen to such a group *without* management. Barring miracles, it would collapse or drift apart very quickly. Things won't simply run themselves, much as we might wish otherwise. But the management, though it may be quite active, is often informal, subtle, and low-key. The only thing keeping a development group together is their shared belief that they can do more in concert than individually. Thus the goal of management is mostly to ensure that they continue to believe this, by setting standards for communications, by making sure useful developers don't get marginalized due to personal idiosyncracies, and in general by making the project a place developers want to keep coming back to. Specific techniques for doing this are discussed throughout the rest of this book.

Finally, there is a general category of problems that may be called "failures of cultural navigation." Ten years ago, even five, it would have been premature to talk about a global culture of free software, but not anymore. A recognizable culture has slowly emerged, and while it is certainly not monolithic—it is at least as prone to internal dissent and factionalism as any geographically bound culture—it does have a basically consistent core. Most successful open source projects exhibit some or all of the characteristics of this core. They reward certain types of behaviors, and punish others; they create an atmosphere that encourages unplanned participation, sometimes at the expense of central coordination; they have concepts of rudeness and politeness that can differ substantially from those prevalent elsewhere. Most importantly, longtime participants have generally internalized these standards, so that they share a rough consensus about expected conduct. Unsuccessful projects usually deviate in significant ways from this core, albeit unintentionally, and often do not have a consensus about what constitutes reasonable default behavior. This means that when problems arise, the situation can quickly deteriorate, as the participants lack an already established stock of cultural reflexes to fall back on for resolving differences.

This book is a practical guide, not an anthropological study or a history. However, a working knowledge of the origins of today's free software culture is an essential foundation for any practical advice. A person who understands the culture can travel far and wide in the open source world, encountering many local variations in custom and dialect, yet still be able to participate comfortably and effectively everywhere. In contrast, a person who does not understand the culture will find the process of organizing or participating in a project difficult and full of surprises. Since the number of people developing free soft-

ware is still growing by leaps and bounds, there are many people in that latter category—this is largely a culture of recent immigrants, and will continue to be so for some time. If you think you might be one of them, the next section provides background for discussions you'll encounter later, both in this book and on the Internet. (On the other hand, if you've been working with open source for a while, you may already know a lot of its history, so feel free to skip the next section.)

History

Software sharing has been around as long as software itself. In the early days of computers, manufacturers felt that competitive advantages were to be had mainly in hardware innovation, and therefore didn't pay much attention to software as a business asset. Many of the customers for these early machines were scientists or technicians, who were able to modify and extend the software shipped with the machine themselves. Customers sometimes distributed their patches back not only to the manufacturer, but to other owners of similar machines. The manufacturers often tolerated and even encouraged this: in their eyes, improvements to the software, from whatever source, just made the machine more attractive to other potential customers.

Although this early period resembled today's free software culture in many ways, it differed in two crucial respects. First, there was as yet little standardization of hardware—it was a time of flourishing innovation in computer design, but the diversity of computing architectures meant that everything was incompatible with everything else. Thus, software written for one machine would generally not work on another. Programmers tended to acquire expertise in a particular architecture or family of architectures (whereas today they would be more likely to acquire expertise in a programming language or family of languages, confident that their expertise will be transferable to whatever computing hardware they happen to find themselves working with). Because a person's expertise tended to be specific to one kind of computer, their accumulation of expertise had the effect of making that computer more attractive to them and their colleagues. It was therefore in the manufacturer's interests for machine-specific code and knowledge to spread as widely as possible.

Second, there was no Internet. Though there were fewer legal restrictions on sharing than today, there were more technical ones: the means of getting data from place to place were inconvenient and cumbersome, relatively speaking. There were some small, local networks, good for sharing information among employees at the same research lab or company. But there remained barriers to overcome if one wanted to share with everyone, no matter where they were. These barriers *were* overcome in many cases. Sometimes different groups made contact with each other independently, sending disks or tapes through land mail, and sometimes the manufacturers themselves served as central clearing houses for patches. It also helped that many of the early computer developers worked at universities, where publishing one's knowledge was expected. But the physical realities of data transmission meant there was always an impedance to sharing, an impedance proportional to the distance (real or organizational) that the software had to travel. Widespread, frictionless sharing, as we know it today, was not possible.

The Rise of Proprietary Software and Free Software

As the industry matured, several interrelated changes occurred simultaneously. The wild diversity of hardware designs gradually gave way to a few clear winners—winners through superior technology, superior marketing, or some combination of the two. At the same time, and not entirely coincidentally, the development of so-called "high level" programming languages meant that one could write a program once, in one language, and have it automatically translated ("compiled") to run on different kinds of computers. The implications of this were not lost on the hardware manufacturers: a customer could now undertake a major software engineering effort without necessarily locking themselves into one particular computer architecture. When this was combined with the gradual narrowing of performance differences between various computers, as the less efficient designs were weeded out, a manufacturer that treated its hardware as its only asset could look forward to a future of declining profit margins. Raw computing power was becoming a fungible good, while software was becoming the differentiator. Selling software, or at least treating it as an integral part of hardware sales, began to look like a good strategy.

This meant that manufacturers had to start enforcing the copyrights on their code more strictly. If users simply continued to share and modify code freely among themselves, they might independently reimplement some of the improvements now being sold as "added value" by the supplier. Worse, shared code could get into the hands of competitors. The irony is that all this was happening around the time the Internet was getting off the ground. Just when truly unobstructed software sharing was finally becoming technically possible, changes in the computer business made it economically undesirable, at least from the point of view of any single company. The suppliers clamped down, either denying users access to the code that ran their machines, or insisting on non-disclosure agreements that made effective sharing impossible.

Conscious resistance

As the world of unrestricted code swapping slowly faded away, a counterreaction crystallized in the mind of at least one programmer. Richard Stallman worked in the Artificial Intelligence Lab at the Massachusetts Institute of Technology in the 1970s and early '80s, during what turned out to be a golden age and a golden location for code sharing. The AI Lab had a strong "hacker ethic",[2] and people were not only encouraged but expected to share whatever improvements they made to the system. As Stallman wrote later:

> *We did not call our software "free software", because that term did not yet exist; but that is what it was. Whenever people from another university or a company wanted to port and use a program, we gladly let them. If you saw someone using an unfamiliar and interesting program, you could always ask to see the source code, so that you could read it, change it, or cannibalize parts of it to make a new program.*
>
> (from **http://www.gnu.org/gnu/thegnuproject.html**)

This Edenic community collapsed around Stallman shortly after 1980, when the changes that had been happening in the rest of the industry finally caught up with the AI Lab. A startup company hired away many of the Lab's programmers to work on an operating system similar to what they had been working on at the Lab, only now under an exclusive license. At the same time, the AI Lab acquired new equipment that came with a proprietary operating system.

Stallman saw the larger pattern in what was happening:

> *The modern computers of the era, such as the VAX or the 68020, had their own operating systems, but none of them were free software: you had to sign a nondisclosure agreement even to get an executable copy.*
>
> *This meant that the first step in using a computer was to promise not to help your neighbor. A cooperating community was forbidden. The rule made by the owners of proprietary software was, "If you share with your neighbor, you are a pirate. If you want any changes, beg us to make them."*

By some quirk of personality, he decided to resist the trend. Instead of continuing to work at the now-decimated AI Lab, or taking a job writing code at one of the new companies, where the results of his work would be kept locked in a box, he resigned from the Lab and started the GNU Project and the Free Software Foundation (FSF). The goal of GNU[3] was to develop a completely free and open computer operating system and body of application software, in which users would never be prevented from hacking or from sharing their modifications. He was, in essence, setting out to recreate what had been destroyed at the AI Lab, but on a world-wide scale and without the vulnerabilities that had made the AI Lab's culture susceptible to disintegration.

[2] Stallman uses the word "hacker" in the sense of "someone who loves to program and enjoys being clever about it," not the relatively new meaning of "someone who breaks into computers."
[3] It stands for "GNU's Not Unix", and the "GNU" in that expansion stands for...the same thing.

In addition to working on the new operating system, Stallman devised a copyright license whose terms guaranteed that his code would be perpetually free. The GNU General Public License (GPL) is a clever piece of legal judo: it says that the code may be copied and modified without restriction, and that both copies and derivative works (i.e., modified versions) must be distributed under the same license as the original, with no additional restrictions. In effect, it uses copyright law to achieve an effect opposite to that of traditional copyright: instead of limiting the software's distribution, it prevents *anyone*, even the author, from limiting it. For Stallman, this was better than simply putting his code into the public domain. If it were in the public domain, any particular copy of it could be incorporated into a proprietary program (as has also been known to happen to code under permissive copyright licenses). While such incorporation wouldn't in any way diminish the original code's continued availability, it would have meant that Stallman's efforts could benefit the enemy—proprietary software. The GPL can be thought of as a form of protectionism for free software, because it prevents non-free software from taking full advantage of GPLed code. The GPL and its relationship to other free software licenses are discussed in detail in Chapter 9, *Licenses, Copyrights, and Patents*.

With the help of many programmers, some of whom shared Stallman's ideology and some of whom simply wanted to see a lot of free code available, the GNU Project began releasing free replacements for many of the most critical components of an operating system. Because of the now-widespread standardization in computer hardware and software, it was possible to use the GNU replacements on otherwise non-free systems, and many people did. The GNU text editor (Emacs) and C compiler (GCC) were particularly successful, gaining large and loyal followings not on ideological grounds, but simply on their technical merits. By about 1990, GNU had produced most of a free operating system, except for the kernel—the part that the machine actually boots up, and that is responsible for managing memory, disk, and other system resources.

Unfortunately, the GNU project had chosen a kernel design that turned out to be harder to implement than expected. The ensuing delay prevented the Free Software Foundation from making the first release of an entirely free operating system. The final piece was put into place instead by Linus Torvalds, a Finnish computer science student who, with the help of volunteers around the world, had completed a free kernel using a more conservative design. He named it Linux, and when it was combined with the existing GNU programs, the result was a completely free operating system. For the first time, you could boot up your computer and do work without using any proprietary software. [4]

Much of the software on this new operating system was not produced by the GNU project. In fact, GNU wasn't even the only group working on producing a free operating system (for example, the code that eventually became NetBSD and FreeBSD was already under development by this time). The importance of the Free Software Foundation was not only in the code they wrote, but in their political rhetoric. By talking about free software as a cause instead of a convenience, they made it difficult for programmers *not* to have a political consciousness about it. Even those who disagreed with the FSF had to engage the issue, if only to stake out a different position. The FSF's effectiveness as propagandists lay in tying their code to a message, by means of the GPL and other texts. As their code spread widely, that message spread as well.

Accidental resistance

There were many other things going on in the nascent free software scene, however, and few were as explictly ideological as Stallman's GNU Project. One of the most important was the *Berkeley Software Distribution* (*BSD*), a gradual re-implementation of the Unix operating system—which up until the late 1970's had been a loosely proprietary research project at AT&T—by programmers at the University of California at Berkeley. The BSD group did not make any overt political statements about the need for programmers to band together and share with one another, but they *practiced* the idea with flair and enthusiasm, by coordinating a massive distributed development effort in which the Unix command-line utilities and code libraries, and eventually the operating system kernel itself, were rewritten from scratch mostly by volunteers. The BSD project became a prime example of non-ideological free software devel-

[4] Technically, Linux was not the first. A free operating system for IBM-compatible computers, called 386BSD, had come out shortly before Linux. However, it was a lot harder to get 386BSD up and running. Linux made such a splash not only because it was free, but because it actually had a high chance of booting your computer when you installed it.

opment, and also served as a training ground for many developers who would go on to remain active in the open source world.

Another crucible of cooperative development was the *X Window System*, a free, network-transparent graphical computing environment, developed at MIT in the mid-1980's in partnership with hardware vendors who had a common interest in being able to offer their customers a windowing system. Far from opposing proprietary software, the X license deliberately allowed proprietary extensions on top of the free core—each member of the consortium wanted the chance to enhance the default X distribution, and thereby gain a competitive advantage over the other members. X Windows[5] itself was free software, but mainly as a way to level the playing field between competing business interests, not out of some desire to end the dominance of proprietary software. Yet another example, predating the GNU project by a few years, was TeX, Donald Knuth's free, publishing-quality typesetting system. He released it under a license that allowed anyone to modify and distribute the code, but not to call the result "TeX" unless it passed a very strict set of compatibility tests (this is an example of the "trademark-protecting" class of free licenses, discussed more in Chapter 9, *Licenses, Copyrights, and Patents*). Knuth wasn't taking a stand one way or the other on the question of free-versus-proprietary software, he just needed a better typesetting system in order to complete his *real* goal—a book on computer programming—and saw no reason not to release his system to the world when done.

Without listing every project and every license, it's safe to say that by the late 1980's, there was a lot of free software available under a wide variety of licenses. The diversity of licenses reflected a corresponding diversity of motivations. Even some of the programmers who chose the GNU GPL were much less ideologically driven than the GNU project itself. Although they enjoyed working on free software, many developers did not consider proprietary software a social evil. There were people who felt a moral impulse to rid the world of "software hoarding" (Stallman's term for non-free software), but others were motivated more by technical excitement, or by the pleasure of working with like-minded collaborators, or even by a simple human desire for glory. Yet by and large these disparate motivations did not interact in destructive ways. This is partly because software, unlike other creative forms like prose or the visual arts, must pass semi-objective tests in order to be considered successful: it must run, and be reasonably free of bugs. This gives all participants in a project a kind of automatic common ground, a reason and a framework for working together without worrying too much about qualifications beyond the technical.

Developers had another reason to stick together as well: it turned out that the free software world was producing some very high-quality code. In some cases, it was demonstrably technically superior to the nearest non-free alternative; in others, it was at least comparable, and of course it always cost less. While only a few people might have been motivated to run free software on strictly philosophical grounds, a great many people were happy to run it because it did a better job. And of those who used it, some percentage were always willing to donate their time and skills to help maintain and improve the software.

This tendency to produce good code was certainly not universal, but it was happening with increasing frequency in free software projects around the world. Businesses that depended heavily on software gradually began to take notice. Many of them discovered that they were already using free software in day-to-day operations, and simply hadn't known it (upper management isn't always aware of everything the IT department does). Corporations began to take a more active and public role in free software projects, contributing time and equipment, and sometimes even directly funding the development of free programs. Such investments could, in the best scenarios, repay themselves many times over. The sponsor only pays a small number of expert programmers to devote themselves to the project full time, but reaps the benefits of *everyone's* contributions, including work from unpaid volunteers and from programmers being paid by other corporations.

"Free" Versus "Open Source"

As the corporate world gave more and more attention to free software, programmers were faced with new issues of presentation. One was the word "free" itself. On first hearing the term "free software"

[5] They prefer it to be called the "X Window System", but in practice, people usually call it "X Windows", because three words is just too cumbersome.

many people mistakenly think it means just "zero-cost software." It's true that all free software is zero-cost,[6] but not all zero-cost software is free. For example, during the battle of the browsers in the 1990s, both Netscape and Microsoft gave away their competing web browsers at no charge, in a scramble to gain market share. Neither browser was free in the "free software" sense. You couldn't get the source code, and even if you could, you didn't have the right to modify or redistribute it.[7] The only thing you could do was download an executable and run it. The browsers were no more free than shrink-wrapped software bought in a store; they merely had a lower price.

This confusion over the word "free" is due entirely to an unfortunate ambiguity in the English language. Most other tongues distinguish low prices from liberty (the distinction between *gratis* and *libre* is immediately clear to speakers of Romance languages, for example). But English's position as the de facto bridge language of the Internet means that a problem with English is, to some degree, a problem for everyone. The misunderstanding around the word "free" was so prevalent that free software programmers eventually evolved a standard formula in response: "It's *free* as in *freedom*—think *free speech*, not *free beer*." Still, having to explain it over and over is tiring. Many programmers felt, with some justification, that the ambiguous word "free" was hampering the public's understanding of this software.

But the problem went deeper than that. The word "free" carried with it an inescapable moral connotation: if freedom was an end in itself, it didn't matter whether free software also happened to be better, or more profitable for certain businesses in certain circumstances. Those were merely pleasant side effects of a motive that was, at bottom, neither technical nor mercantile, but moral. Furthermore, the "free as in freedom" position forced a glaring inconsistency on corporations who wanted to support particular free programs in one aspect of their business, but continue marketing proprietary software in others.

These dilemmas came to a community that was already poised for an identity crisis. The programmers who actually *write* free software have never been of one mind about the overall goal, if any, of the free software movement. Even to say that opinions run from one extreme to the other would be misleading, in that it would falsely imply a linear range where there is instead a multidimensional scattering. However, two broad categories of belief can be distinguished, if we are willing to ignore subtleties for the moment. One group takes Stallman's view, that the freedom to share and modify is the most important thing, and that therefore if you stop talking about freedom, you've left out the core issue. Others feel that the software itself is the most important argument in its favor, and are uncomfortable with proclaiming proprietary software inherently bad. Some, but not all, free software programmers believe that the author (or employer, in the case of paid work) *should* have the right to control the terms of distribution, and that no moral judgement need be attached to the choice of particular terms.

For a long time, these differences did not need to be carefully examined or articulated, but free software's burgeoning success in the business world made the issue unavoidable. In 1998, the term *open source* was created as an alternative to "free", by a coalition of programmers who eventually became The Open Source Initiative (OSI).[8] The OSI felt not only that "free software" was potentially confusing, but that the word "free" was just one symptom of a general problem: that the movement needed a marketing program to pitch it to the corporate world, and that talk of morals and the social benefits of sharing would never fly in corporate boardrooms. In their own words:

> *The Open Source Initiative is a marketing program for free software. It's a pitch for "free software" on solid pragmatic grounds rather than ideological tub-thumping. The winning substance has not changed, the losing attitude and symbolism have. ...*
>
> *The case that needs to be made to most techies isn't about the concept of open source, but the name. Why not call it, as we traditionally have, free software?*
>
> *One direct reason is that the term "free software" is easily misunderstood in ways that lead to conflict. ...*

[6] One may charge a fee for giving out copies of free software, but since one cannot stop the recipients from offering it at no charge afterwards, the price is effectively driven to zero immediately.
[7] The source code to Netscape Navigator *was* eventually released under an open source license, in 1998, and became the foundation for the Mozilla web browser. See http://www.mozilla.org/.
[8] OSI's web home is http://www.opensource.org/.

> *But the real reason for the re-labeling is a marketing one. We're trying to pitch our concept to the corporate world now. We have a winning product, but our positioning, in the past, has been awful. The term "free software" has been misunderstood by business persons, who mistake the desire to share with anti-commercialism, or worse, theft.*
>
> *Mainstream corporate CEOs and CTOs will never buy "free software." But if we take the very same tradition, the same people, and the same free-software licenses and change the label to "open source" ? that, they'll buy.*
>
> *Some hackers find this hard to believe, but that's because they're techies who think in concrete, substantial terms and don't understand how important image is when you're selling something.*
>
> *In marketing, appearance is reality. The appearance that we're willing to climb down off the barricades and work with the corporate world counts for as much as the reality of our behavior, our convictions, and our software.*
>
> (from **http://opensource.feratech.com/advocacy/faq.php** and **http://opensource.feratech.com/advocacy/case_for_hackers.php#marketing**)

The tips of many icebergs of controversy are visible in that text. It refers to "our convictions", but smartly avoids spelling out exactly what those convictions are. For some, it might be the conviction that code developed according to an open process will be better code; for others, it might be the conviction that all information should be shared. There's the use of the word "theft" to refer (presumably) to illegal copying—a usage that many object to, on the grounds that it's not theft if the original possessor still has the item afterwards. There's the tantalizing hint that the free software movement might be mistakenly accused of anti-commercialism, but it leaves carefully unexamined the question of whether such an accusation would have any basis in fact.

None of which is to say that the OSI's web site is inconsistent or misleading. It's not. Rather, it is an example of exactly what the OSI claims had been missing from the free software movement: good marketing, where "good" means "viable in the business world." The Open Source Initiative gave a lot of people exactly what they had been looking for—a vocabulary for talking about free software as a development methodology and business strategy, instead of as a moral crusade.

The appearance of the Open Source Initiative changed the landscape of free software. It formalized a dichotomy that had long been unnamed, and in doing so forced the movement to acknowledge that it had internal politics as well as external. The effect today is that both sides have had to find common ground, since most projects include programmers from both camps, as well as participants who don't fit any clear category. This doesn't mean people never talk about moral motivations—lapses in the traditional "hacker ethic" are sometimes called out, for example. But it is rare for a free software / open source developer to openly question the basic motivations of others in a project. The contribution trumps the contributor. If someone writes good code, you don't ask them whether they do it for moral reasons, or because their employer paid them to, or because they're building up their resumé, or whatever. You evaluate the contribution on technical grounds, and respond on technical grounds. Even explicitly political organizations like the Debian project, whose goal is to offer a 100% free (that is, "free as in freedom") computing environment, are fairly relaxed about integrating with non-free code and cooperating with programmers who don't share exactly the same goals.

The Situation Today

When running a free software project, you won't need to talk about such weighty philosophical matters on a daily basis. Programmers will not insist that everyone else in the project agree with their views on all things (those who do insist on this quickly find themselves unable to work in any project). But you do need to be aware that the question of "free" versus "open source" exists, partly to avoid saying things

that might be inimical to some of the participants, and partly because understanding developers' motivations is the best way—in some sense, the *only* way—to manage a project.

Free software is a culture by choice. To operate successfully in it, you have to understand why people choose to be in it in the first place. Coercive techniques don't work. If people are unhappy in one project, they will just wander off to another one. Free software is remarkable even among volunteer communities for its lightness of investment. Most of the people involved have never actually met the other participants face-to-face, and simply donate bits of time whenever they feel like it. The normal conduits by which humans bond with each other and form lasting groups are narrowed down to a tiny channel: the written word, carried over electronic wires. Because of this, it can take a long time for a cohesive and dedicated group to form. Conversely, it's quite easy for a project to lose a potential volunteer in the first five minutes of acquaintanceship. If a project doesn't make a good first impression, newcomers rarely give it a second chance.

The transience, or rather the *potential* transience, of relationships is perhaps the single most daunting task facing a new project. What will persuade all these people to stick together long enough to produce something useful? The answer to that question is complex enough to occupy the rest of this book, but if it had to be expressed in one sentence, it would be this:

> *People should feel that their connection to a project, and influence over it, is directly proportional to their contributions.*

No class of developers, or potential developers, should ever feel discounted or discriminated against for non-technical reasons. Clearly, projects with corporate sponsorship and/or salaried developers need to be especially careful in this regard, as Chapter 5, *Money* discusses in detail. Of course, this doesn't mean that if there's no corporate sponsorship then you have nothing to worry about. Money is merely one of many factors that can affect the success of a project. There are also questions of what language to choose, what license, what development process, precisely what kind of infrastructure to set up, how to publicize the project's inception effectively, and much more. Starting a project out on the right foot is the topic of the next chapter.